ArcFlow

A self-custody reference dApp on Arc Testnet

Payment · Swap · Staking · Bridge — built end-to-end on Circle App Kit & CCTP

A community contribution to the Arc ecosystem

🌐 arcflow.click · ⛓️ chainId 5042002

Why we built this

Arc is a new chain. New chains grow when developers can answer one question fast:

> *"Can I build a real, full-stack app on this — wallet, payments, swap, staking, cross-chain — today?"*

ArcFlow is our answer: yes. It's a complete, working dApp that exercises the core Circle/Arc stack, and a transparent reference other builders can learn from.

We're sharing it with the Arc team in the hope it's useful as:

- a **public reference implementation** for the four most-asked-for flows,
- **honest developer-experience feedback** from actually shipping on Arc Testnet.

What ArcFlow does — 4 modules + 2 surfaces

Module	What the user does	How it's powered
Payment	Buy a demo package, pay in USDC	Direct ERC-20 transfer, verified on-chain
Swap	Same-chain swap USDC ↔ EURC ↔ cirBTC	Circle **App Kit** `kit.swap`
Staking	Stake USDC/EURC, earn USDC rewards	Custom **ArcStaking** Solidity contract
Bridge	Move USDC across chains	Circle **CCTP V2** via `kit.bridge`
History	Unified personal activity feed	Backend indexer/verifier
Admin	Aggregate metrics dashboard	SIWE-gated (Sign-In-With-Ethereum)

Core principle: self-custody, no shortcuts

- The browser **never holds a private key**. The user signs **every** transaction in MetaMask.
- The backend is **read-only on-chain**: it verifies tx receipts and records state — it can't move funds.
- The operator key is used **only** by the contracts package (deploy + fund the staking pool) and a read-only indexer — **never** in the frontend.

> This is the security posture we'd want every Arc dApp to start from. ArcFlow shows it's achievable without custodial backends.

Architecture

Three packages: **contracts/** (Hardhat) · **backend/** (API + indexer) · **frontend/** (Next.js). One shared config drives all three.

The Circle / Arc stack we exercised

- **App Kit** (`@circle-fin/app-kit` + `@circle-fin/adapter-viem-v2`)
  - Browser adapter over the wallet's **EIP-1193** provider — `createViemAdapterFromProvider` , *not* the private-key adapter. User stays in control.
  - One API for `send` , `swap` , `bridge` .
- **CCTP V2** for native USDC bridging
  - Domain **26**, TokenMessenger / MessageTransmitter / TokenMinter V2 wired from config.
- **USDC as the native gas token** on Arc (18 decimals) — a genuinely novel UX we designed around.
- **Tokens:** USDC, EURC, cirBTC (real Arc Testnet addresses).

Live deployment — it's running, not a mock

- **Network:** Arc Testnet · chainId **5042002** · RPC `rpc.testnet.arc.network` · explorer `testnet.arcscan.app`
- **ArcStaking:** `0x6e79287f62B47307B5b7349072c51AdAF8833d54` (reward pool funded **10 USDC**, fixed **10% APY**, per-second linear)
- **Payment receiver / demo packages:** Starter 1 / Pro 5 / Premium 10 USDC, 30-min order expiry
- **Hosting:** Docker Compose, exposed via **Cloudflare Tunnel** (no VPS) at `arcflow.click` (+ `api.arcflow.click`)

> Connect MetaMask, switch to Arc Testnet, and run a real payment / swap / stake / bridge.

Developer-experience feedback for the Arc team

The most valuable thing we can give back: **what we hit while building.** All solvable, but worth documenting for the next builder.

1. **USDC-as-gas has two decimal worlds.** Native gas USDC is 18-decimal; ERC-20 USDC is 6-decimal. Easy to mix up in fee estimates — clearer docs / a helper would save time.
2. **cirBTC swap pool is thin & mispriced.** The on-chain execution price diverges hard from the App Kit quote; default 0.5% slippage reverts. We auto-raise to 50% for any cirBTC pair. More seeded testnet liquidity would make demos smoother.
3. **CCTP bridge completion is async.** We record source confirmation but destination mint needs an attestation poller — a small reference "CCTP status poller" in Arc docs would help.

What works really well on Arc (the good news)

- **App Kit browser adapter is clean.** One `createViemAdapterFromProvider` call and `send` / `swap` / `bridge` just work over MetaMask — no custom routing contracts needed for payment, swap, or bridge.
- **Standard EVM tooling is fully compatible.** Hardhat deploy, `viem` public client, wagmi connectors — zero Arc-specific glue beyond chain config.
- **CCTP V2 native USDC bridging** to Ethereum/Base Sepolia worked with config-only setup.
- **Fast path to a real product:** four production-shaped flows in a single small codebase.

How ArcFlow can help Arc going forward

- **Open reference dApp** the Arc team can point new builders to (self-custody pattern, App Kit usage, CCTP wiring, on-chain verification backend).
- **A living DX test harness** — we can re-run it against new Arc / App Kit releases and report regressions.
- **Feedback loop** — happy to file detailed issues on the decimals/liquidity/attestation items above and help refine onboarding docs.
- **Roadmap if useful to Arc:** CCTP destination poller, more bridge tokens (EURC), richer payment/checkout demo, mainnet-ready hardening.

What we'd love from the Arc team

- A quick look at the **DX feedback** (slides on decimals / cirBTC liquidity / CCTP attestation) — confirm or correct our understanding.
- **Seeded testnet liquidity** for cirBTC pairs so swaps demo cleanly.
- Guidance on **App Kit / CCTP roadmap** so we align ArcFlow with where Arc is heading.
- If valuable: list ArcFlow as a **community reference** for Arc builders.

Thank you

ArcFlow — built on Arc, for Arc builders.

🌐 **arcflow.click** · ⛓️ ArcStaking `0x6e79…3d54` · 📦 Payment · Swap · Staking · Bridge

Self-custody. Standard EVM tooling. Circle App Kit + CCTP. Running on Arc Testnet today.

We'd be glad to demo it live and contribute our findings back to Arc.

Appendix — backend API surface

Public: `GET /api/health` · `GET /api/config` · `POST /api/orders` + `/:id` + `/:id/pay` · `GET /api/history?address=` · record endpoints `POST /api/swaps | /bridges | /stakes` (each verified on-chain before stored).

Admin (SIWE): `nonce` → `verify` → session cookie · `GET /api/admin/overview | /payments | /swaps | /staking | /bridges`.

Statuses tracked: payment (pending→paid/failed/expired) · swap (draft→success/failed) · staking (no_stake→active→claimable) · bridge (pending_source→completed/failed).

Appendix — ArcStaking contract

Solidity `^0.8.24`. Stake an ERC-20 (USDC/EURC), earn **USDC** rewards.

- `stake(token, amount)` · `unstake(token, amount)` · `claim(token)`
- Views: `pendingReward(user, token)` · `stakeInfo(user, token)`
- Reward = `principal × apyBps/10000 × elapsed / 365d` (linear, per-second)
- Config-driven: `apyBps` (1000 = 10%), `lockSeconds` (0), `minStake` (1.00), `rewardPoolFund` (10 USDC)
- Emits `Staked` / `Unstaked` / `Claimed` (indexer-friendly)

> The only custom contract in the whole project — everything else rides Circle App Kit.

Appendix — disclaimer (testnet)

> This is a testnet demo. All tokens — testnet USDC, EURC, cirBTC — have **no real-world value**. ArcFlow is **not** a payment service, exchange, investment product, or yield product, and provides no real financial return.

Shown as a persistent banner in-app; every action previews token, amount, network, fee/slippage and a testnet warning before signing, then links to the explorer.